

```
import { useState, useEffect, useRef, useCallback } from "react";
import { LineChart, Line, XAxis, YAxis, Tooltip, ResponsiveContainer, ReferenceLine } from
"recharts";

// — Deriv WebSocket API —————
const WS_URL = "wss://ws.binaryws.com/websockets/v3?app_id=1089";

function useDerivWS(token) {
  const ws = useRef(null);
  const listeners = useRef({});
  const [connected, setConnected] = useState(false);
  const [error, setError] = useState(null);

  const send = useCallback((msg) => {
    if (ws.current && ws.current.readyState === WebSocket.OPEN) {
      ws.current.send(JSON.stringify(msg));
    }
  }, []);

  const on = useCallback((key, fn) => { listeners.current[key] = fn; }, []);
  const off = useCallback((key) => { delete listeners.current[key]; }, []);

  useEffect(() => {
    if (!token) return;
    const socket = new WebSocket(WS_URL);
    ws.current = socket;

    socket.onopen = () => {
      setConnected(true);
      setError(null);
      socket.send(JSON.stringify({ authorize: token }));
    };
    socket.onmessage = (e) => {
      const data = JSON.parse(e.data);
      const type = data.msg_type;
      if (listeners.current[type]) listeners.current[type](data);
      if (listeners.current["*"]) listeners.current["*"](data);
    };
    socket.onerror = () => setError("WebSocket error. Check your token.");
    socket.onclose = () => setConnected(false);

    return () => { socket.close(); };
  }, [token]);

  return { send, on, off, connected, error };
}

// — Symbols —————
```

```

const SYMBOLS = [
  { id: "R_100",    label: "Volatility 100" },
  { id: "R_75",     label: "Volatility 75"  },
  { id: "R_50",     label: "Volatility 50"   },
  { id: "frxEURUSD", label: "EUR/USD"        },
  { id: "frxGBPUSD", label: "GBP/USD"        },
  { id: "frxAUUSD",  label: "Gold/USD"       },
];

// — Styles —
const styles = `
  @import url('https://fonts.googleapis.com/css2?
family=Space+Mono:ital,wght@0,400;0,700;1,400&family=Syne:wght@400;600;800&display=swap');
  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
  :root {
    --bg: #080c12; --surface: #0e1520; --card: #121c2b;
    --border: #1e2d42; --accent: #00e5c4; --accent2: #ff4d6d;
    --accent3: #f7c948; --muted: #4a637f; --text: #d0e4f7;
    --textDim: #6b8aaa; --up: #00e5c4; --down: #ff4d6d;
    --font: 'Space Mono', monospace; --display: 'Syne', sans-serif;
  }
  body { background: var(--bg); color: var(--text); font-family: var(--font); font-size: 13px; }
  .app {
    min-height: 100vh; display: grid; grid-template-rows: 56px 1fr;
    background: var(--bg);
    background-image: radial-gradient(ellipse 80% 40% at 50% -10%, #00e5c4 15 0%, transparent
70%);
  }
  .topbar {
    display: flex; align-items: center; justify-content: space-between;
    padding: 0 24px; border-bottom: 1px solid var(--border);
    background: var(--surface);
  }
  .topbar-logo {
    font-family: var(--display); font-size: 20px; font-weight: 800;
    color: var(--accent); letter-spacing: -0.5px;
    display: flex; align-items: center; gap: 8px;
  }
  .topbar-logo span { color: var(--text); }
  .topbar-right { display: flex; align-items: center; gap: 12px; }
  .status-dot {
    width: 8px; height: 8px; border-radius: 50%;
    background: var(--muted); transition: all 0.3s;
  }
  .status-dot.live { background: var(--up); box-shadow: 0 0 8px var(--up); animation: pulse 2s
infinite; }
  @keyframes pulse { 0%,100%{opacity:1} 50%{opacity:0.5} }
  .token-form { display: flex; gap: 8px; }
  .token-input {
    background: var(--card); border: 1px solid var(--border);

```

```
color: var(--text); font-family: var(--font); font-size: 11px;
padding: 6px 12px; border-radius: 4px; width: 240px; outline: none;
transition: border 0.2s;
}
.token-input:focus { border-color: var(--accent); }
.btn {
  background: var(--accent); color: var(--bg);
  font-family: var(--font); font-size: 11px; font-weight: 700;
  padding: 6px 14px; border: none; border-radius: 4px; cursor: pointer;
  transition: opacity 0.2s;
}
.btn:hover { opacity: 0.85; }
.btn.danger { background: var(--accent2); }
.main { display: grid; grid-template-columns: 220px 1fr 300px; height: calc(100vh - 56px);
overflow: hidden; }
.sidebar {
  border-right: 1px solid var(--border); background: var(--surface);
  overflow-y: auto; display: flex; flex-direction: column;
}
.sidebar-section { padding: 16px; border-bottom: 1px solid var(--border); }
.sidebar-label { font-size: 10px; color: var(--muted); letter-spacing: 2px; text-transform:
uppercase; margin-bottom: 10px; }
.symbol-item {
  padding: 8px 10px; border-radius: 4px; cursor: pointer;
  display: flex; justify-content: space-between; align-items: center;
  transition: background 0.15s; margin-bottom: 2px;
}
.symbol-item:hover { background: var(--card); }
.symbol-item.active { background: var(--card); border-left: 2px solid var(--accent); padding-
left: 8px; }
.symbol-name { font-size: 12px; color: var(--text); }
.symbol-price { font-size: 11px; color: var(--accent); font-weight: 700; }
.center { display: flex; flex-direction: column; overflow: hidden; }
.stats-bar {
  display: grid; grid-template-columns: repeat(4,1fr);
  gap: 1px; background: var(--border);
  border-top: 1px solid var(--border); border-bottom: 1px solid var(--border);
}
.stat-cell { background: var(--surface); padding: 10px 16px; display: flex; flex-direction:
column; gap: 2px; }
.stat-label { font-size: 9px; color: var(--muted); text-transform: uppercase; letter-spacing:
1.5px; }
.stat-val { font-size: 14px; font-weight: 700; color: var(--text); }
.stat-val.green { color: var(--up); }
.stat-val.red { color: var(--down); }
.chart-area { flex: 1; padding: 20px; display: flex; flex-direction: column; gap: 12px;
overflow: hidden; }
.chart-header { display: flex; justify-content: space-between; align-items: flex-start; }
.chart-symbol { font-family: var(--display); font-size: 22px; font-weight: 800; color: var(--
text); }
```

```
.chart-price { font-family: var(--display); font-size: 32px; font-weight: 800; color: var(--accent); transition: color 0.3s; }
.chart-price.down { color: var(--down); }
.chart-change { font-size: 11px; margin-top: 2px; }
.chart-change.up { color: var(--up); }
.chart-change.down { color: var(--down); }
.chart-box {
  flex: 1; background: var(--card); border: 1px solid var(--border); border-radius: 8px;
  padding: 12px 4px 4px 0; min-height: 0;
}
.trade-panel {
  border-left: 1px solid var(--border); background: var(--surface);
  display: flex; flex-direction: column; overflow-y: auto;
}
.panel-tabs { display: flex; border-bottom: 1px solid var(--border); }
.panel-tab {
  flex: 1; padding: 14px; text-align: center;
  font-size: 10px; font-weight: 700; letter-spacing: 1px; text-transform: uppercase;
  cursor: pointer; color: var(--muted); border-bottom: 2px solid transparent; transition: all
0.2s;
}
.panel-tab.active { color: var(--accent); border-bottom-color: var(--accent); }
.panel-body { padding: 16px; display: flex; flex-direction: column; gap: 14px; }
.form-group { display: flex; flex-direction: column; gap: 6px; }
.form-label { font-size: 10px; color: var(--muted); letter-spacing: 1.5px; text-transform:
uppercase; }
.form-input, .form-select {
  background: var(--card); border: 1px solid var(--border);
  color: var(--text); font-family: var(--font); font-size: 12px;
  padding: 8px 12px; border-radius: 4px; outline: none; width: 100%; transition: border 0.2s;
}
.form-input:focus, .form-select:focus { border-color: var(--accent); }
.form-select option { background: var(--card); }
.contract-btns { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
.contract-btn {
  padding: 14px; border-radius: 6px; border: 1px solid var(--border);
  cursor: pointer; font-family: var(--font); font-size: 12px; font-weight: 700;
  transition: all 0.2s; background: var(--card); color: var(--textDim);
  display: flex; flex-direction: column; align-items: center; gap: 4px;
}
.contract-btn:hover { border-color: var(--muted); }
.contract-btn.call { border-color: var(--up); color: var(--up); background: #00e5c410; }
.contract-btn.put { border-color: var(--down); color: var(--down); background: #ff4d6d10; }
.contract-btn .arrow { font-size: 20px; }
.payout-preview {
  background: var(--card); border: 1px solid var(--border); border-radius: 6px;
  padding: 12px; display: flex; justify-content: space-between;
}
.payout-label { font-size: 10px; color: var(--muted); }
.payout-val { font-size: 16px; font-weight: 700; color: var(--accent3); }
```

```

.open-contracts { padding: 16px; display: flex; flex-direction: column; gap: 8px; }
.contract-card {
  background: var(--card); border: 1px solid var(--border); border-radius: 6px;
  padding: 12px; display: flex; flex-direction: column; gap: 8px;
}
.contract-card-header { display: flex; justify-content: space-between; align-items: center; }
.contract-tag { font-size: 10px; font-weight: 700; letter-spacing: 1px; padding: 2px 8px;
border-radius: 3px; }
.contract-tag.call { background: #00e5c420; color: var(--up); }
.contract-tag.put { background: #ff4d6d20; color: var(--down); }
.contract-pnl { font-size: 14px; font-weight: 700; }
.contract-pnl.pos { color: var(--up); }
.contract-pnl.neg { color: var(--down); }
.contract-details { display: grid; grid-template-columns: 1fr 1fr; gap: 4px; }
.cd-item { font-size: 10px; color: var(--textDim); }
.cd-val { font-size: 11px; color: var(--text); }
.history-list { padding: 16px; display: flex; flex-direction: column; gap: 6px; }
.history-item {
  background: var(--card); border: 1px solid var(--border); border-radius: 6px;
  padding: 10px 12px; display: flex; justify-content: space-between; align-items: center;
}
.history-left { display: flex; flex-direction: column; gap: 3px; }
.history-sym { font-size: 12px; font-weight: 700; color: var(--text); }
.history-meta { font-size: 10px; color: var(--textDim); }
.history-profit { font-size: 13px; font-weight: 700; }
.history-profit.pos { color: var(--up); }
.history-profit.neg { color: var(--down); }
.account-panel { padding: 16px; display: flex; flex-direction: column; gap: 12px; }
.account-card { background: var(--card); border: 1px solid var(--border); border-radius: 8px;
padding: 16px; }
.account-name { font-family: var(--display); font-size: 18px; font-weight: 700; margin-bottom:
4px; }
.account-balance { font-size: 28px; font-weight: 700; color: var(--accent); font-family: var(-
display); }
.account-currency { font-size: 12px; color: var(--muted); margin-top: 2px; }
.account-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
.account-stat { background: var(--surface); border: 1px solid var(--border); border-radius:
6px; padding: 10px; }
.account-stat-label { font-size: 9px; color: var(--muted); text-transform: uppercase; letter-
spacing: 1.5px; margin-bottom: 4px; }
.account-stat-val { font-size: 14px; font-weight: 700; color: var(--text); }
.empty { text-align: center; padding: 40px 20px; color: var(--muted); font-size: 12px; line-
height: 1.8; }
.toast {
  position: fixed; bottom: 24px; right: 24px;
  background: var(--card); border: 1px solid var(--accent);
  color: var(--text); padding: 12px 20px; border-radius: 6px;
  font-size: 12px; z-index: 1000; animation: slideIn 0.3s ease;
}
.toast.error { border-color: var(--down); }

```

```

@keyframes slideIn {from{transform:translateY(20px);opacity:0}
to{transform:translateY(0);opacity:1} }
::-webkit-scrollbar { width: 4px; }
::-webkit-scrollbar-track { background: transparent; }
::-webkit-scrollbar-thumb { background: var(--border); border-radius: 2px; }
`;

const CustomTooltip = ({ active, payload }) => {
  if (!active || !payload?.length) return null;
  return (
    <div style={{ background:"var(--card)", border:"1px solid var(--border)", padding:"8px 12px", borderRadius:4, fontFamily:"var(--font)", fontSize:11 }}>
      <div style={{ color:"var(--accent)" }}>{payload[0]?.value?.toFixed(4)}</div>
      <div style={{ color:"var(--muted)" }}>{payload[0]?.payload?.time}</div>
    </div>
  );
};

export default function App() {
  const [token, setToken] = useState("");
  const [tokenInput, setTokenInput] = useState("");
  const [account, setAccount] = useState(null);
  const [activeSymbol, setActiveSymbol] = useState(SYMBOLS[0]);
  const [prices, setPrices] = useState({});
  const [chartData, setChartData] = useState([]);
  const [activeTab, setActiveTab] = useState("trade");
  const [stake, setStake] = useState("10");
  const [duration, setDuration] = useState("5");
  const [durationType, setDurationType] = useState("t");
  const [openContracts, setOpenContracts] = useState([]);
  const [history, setHistory] = useState([]);
  const [toast, setToast] = useState(null);

  const { send, on, off, connected, error } = useDerivWS(token);

  const showToast = (msg, type = "info") => {
    setToast({ msg, type });
    setTimeout(() => setToast(null), 3500);
  };

  useEffect(() => {
    on("authorize", (data) => {
      if (data.error) { showToast(data.error.message, "error"); return; }
      setAccount(data.authorize);
      showToast(`Connected as ${data.authorize.loginid}`);
      SYMBOLS.forEach(s => send({ ticks: s.id, subscribe: 1 }));
      send({ profit_table: 1, description: 1, limit: 20 });
      send({ portfolio: 1, contract_type: ["CALL","PUT"] });
    });
    on("tick", (data) => {

```

```

    if (!data.tick) return;
    const { symbol, quote, epoch } = data.tick;
    setPrices(p => ({ ...p, [symbol]: { quote, epoch } }));
    if (symbol === activeSymbol.id) {
      const t = new Date(epoch * 1000);
      const time = t.toLocaleTimeString([], { hour:"2-digit", minute:"2-digit", second:"2-digit" });
      setChartData(d => [...d.slice(-80), { time, price: quote }]);
    }
  });
  on("profit_table", (data) => {
    if (data.profit_table?.transactions) setHistory(data.profit_table.transactions);
  });
  on("portfolio", (data) => {
    if (data.portfolio?.contracts) setOpenContracts(data.portfolio.contracts);
  });
  on("buy", (data) => {
    if (data.error) { showToast(data.error.message, "error"); return; }
    showToast(`Contract #${data.buy.contract_id} placed!`);
    send({ portfolio: 1, contract_type: ["CALL", "PUT"] });
  });
  return () => { off("authorize"); off("tick"); off("profit_table"); off("portfolio");
off("buy"); };
}, [on, off, send, activeSymbol]);

useEffect(() => { setChartData([]); }, [activeSymbol]);

const handleConnect = () => { if (tokenInput.trim()) setToken(tokenInput.trim()); };
const handleDisconnect = () => {
  setToken(""); setTokenInput(""); setAccount(null);
  setPrices({}); setChartData([]); setOpenContracts([]); setHistory([]);
};
const placeTrade = (type) => {
  if (!account) { showToast("Connect your API token first.", "error"); return; }
  send({
    buy: 1, price: parseFloat(stake),
    parameters: {
      contract_type: type, symbol: activeSymbol.id,
      duration: parseInt(duration), duration_unit: durationType,
      currency: account.currency, basis: "stake", amount: parseFloat(stake),
    }
  });
};

const currentPrice = prices[activeSymbol.id]?.quote;
const prevChartPrice = chartData.length > 1 ? chartData[chartData.length - 2]?.price : null;
const isUp = currentPrice && prevChartPrice ? currentPrice >= prevChartPrice : true;
const isVolatility = activeSymbol.id.startsWith("R_");
const decimals = isVolatility ? 2 : 5;
const estimatedPayout = (parseFloat(stake || 0) * 1.85).toFixed(2);

```

```

return (
  <>
    <style>{styles}</style>
    <div className="app">
      {/* TOPBAR */}
      <div className="topbar">
        <div className="topbar-logo">◈ <span>DERIV</span>TERMINAL</div>
        <div className="topbar-right">
          <div className={`status-dot ${connected ? "live" : ""}`} />
          <span style={{ fontSize:10, color: connected ? "var(--up)" : "var(--muted)",
letterSpacing:1 }}>
            {connected ? "LIVE" : "OFFLINE"}
          </span>
          {!token ? (
            <div className="token-form">
              <input
                className="token-input"
                placeholder="Paste your Deriv API token..."
                value={tokenInput}
                onChange={e => setTokenInput(e.target.value)}
                onKeyDown={e => e.key === "Enter" && handleConnect()}
                type="password"
              />
              <button className="btn" onClick={handleConnect}>Connect</button>
            </div>
          ) : (
            <div style={{ display:"flex", alignItems:"center", gap:10 }}>
              <span style={{ fontSize:11, color:"var(--textDim)" }}>
                {account?.loginid} · {account?.currency} {account?.balance?.toFixed(2)}
              </span>
              <button className="btn danger" onClick={handleDisconnect}>Disconnect</button>
            </div>
          )}
        </div>
      </div>

      {/* MAIN */}
      <div className="main">
        {/* SIDEBAR */}
        <div className="sidebar">
          <div className="sidebar-section">
            <div className="sidebar-label">Markets</div>
            {SYMBOLS.map(s => (
              <div key={s.id} className={`symbol-item ${activeSymbol.id === s.id ? "active" :
""}`} `} onClick={() => setActiveSymbol(s)}>
                <span className="symbol-name">{s.label}</span>
                <span className="symbol-price">
{prices[s.id]?.quote?.toFixed(s.id.startsWith("R_") ? 2 : 5) || "-"}</span>
              </div>
            ))}
          </div>
        </div>
      </div>
    </div>
  )
}

```



```

    ))}
  </div>
  {account && (
    <div className="sidebar-section">
      <div className="sidebar-label">Account</div>
      <div style={{ display:"flex", flexDirection:"column", gap:6 }}>
        {[["Balance", `${account.currency} ${account.balance?.toFixed(2)}`, "var(--
accent)"]],
          ["Login ID", account.loginid, "var(--text)"],
          ["Country", account.country?.toUpperCase() || "-", "var(--text)"]
        ].map(([label, val, color]) => (
          <div key={label} style={{ display:"flex", justifyContent:"space-between" }}>
            <span style={{ color:"var(--muted)", fontSize:11 }}>{label}</span>
            <span style={{ fontSize:11, color, fontWeight: label === "Balance" ? 700 :
400 }}>{val}</span>
          </div>
        )))
      </div>
    </div>
  )}
  <div className="sidebar-section">
    <div className="sidebar-label">Open Positions</div>
    <div style={{ color: openContracts.length ? "var(--up)" : "var(--muted)",
fontSize:12, fontWeight:700 }}>
      {openContracts.length} active
    </div>
  </div>
</div>

{/* CENTER */}
<div className="center">
  <div className="stats-bar">
    {[
      ["Symbol", activeSymbol.label, ""],
      ["Price", currentPrice?.toFixed(decimals) || "-", isUp ? "green" : "red"],
      ["Open Positions", openContracts.length, ""],
      ["Trade History", history.length, ""],
    ].map(([label, val, cls]) => (
      <div className="stat-cell" key={label}>
        <span className="stat-label">{label}</span>
        <span className={`stat-val ${cls}`}>{val}</span>
      </div>
    )))
  </div>
  <div className="chart-area">
    <div className="chart-header">
      <div>
        <div className="chart-symbol">{activeSymbol.label}</div>
        <div style={{ fontSize:11, color:"var(--muted)", marginTop:2 }}>
{activeSymbol.id}</div>

```

```

    </div>
    <div style={{ textAlign:"right" }}>
      <div className={`chart-price ${!isUp ? "down" : ""}`}>
{currentPrice?.toFixed(decimals) || "-"}</div>
      <div className={`chart-change ${isUp ? "up" : "down"}`}>
        {isUp ? "▲" : "▼"} {prevChartPrice && currentPrice ? Math.abs(currentPrice
- prevChartPrice).toFixed(decimals + 2) : "-"}
      </div>
    </div>
  </div>
  <div className="chart-box">
    {chartData.length > 1 ? (
      <ResponsiveContainer width="100%" height="100%">
        <LineChart data={chartData} margin={{ top:4, right:16, bottom:4, left:10 }}>
          <XAxis dataKey="time" tick={{ fill:"#4a637f", fontSize:9,
fontFamily:"Space Mono" }} tickLine={false} axisLine={false} interval=
{Math.floor(chartData.length / 6)} />
          <YAxis domain={["auto","auto"]} tick={{ fill:"#4a637f", fontSize:9,
fontFamily:"Space Mono" }} tickLine={false} axisLine={false} width={60} tickFormatter={v =>
v.toFixed(decimals - 1)} />
          <Tooltip content={<CustomTooltip />} />
          {currentPrice && <ReferenceLine y={currentPrice} stroke="#00e5c430"
strokeDasharray="4 4" />}
          <Line type="monotone" dataKey="price" stroke={isUp ? "var(--up)" : "var(--
down)"} strokeWidth={2} dot={false} activeDot={{ r:4 }} isAnimationActive={false} />
        </LineChart>
      </ResponsiveContainer>
    ) : (
      <div className="empty">{connected ? "Waiting for tick data..." : "Connect your
API token to see live prices"}</div>
    )}
  </div>
</div>

{/* RIGHT PANEL */}
<div className="trade-panel">
  <div className="panel-tabs">
    {[["trade","Trade"],["positions","Open"],["history","History"],
["account","Account"]].map(([id,label]) => (
      <div key={id} className={`panel-tab ${activeTab === id ? "active" : ""}`}
onClick={() => setActiveTab(id)}>{label}</div>
    ))}
  </div>

  {activeTab === "trade" && (
    <div className="panel-body">
      <div className="form-group">
        <label className="form-label">Symbol</label>
        <select className="form-select" value={activeSymbol.id} onChange={e =>

```

```

setActiveSymbol(SYMBOLS.find(s => s.id === e.target.value)))}}>
    {SYMBOLS.map(s => <option key={s.id} value={s.id}>{s.label}</option>)}
  </select>
</div>
<div className="form-group">
  <label className="form-label">Stake (USD)</label>
  <input className="form-input" type="number" value={stake} onChange={e =>
setStake(e.target.value)} min="1" step="1" />
</div>
<div className="form-group">
  <label className="form-label">Duration</label>
  <div style={{ display:"grid", gridTemplateColumns:"2fr 1fr", gap:8 }}>
    <input className="form-input" type="number" value={duration} onChange={e =>
setDuration(e.target.value)} min="1" />
    <select className="form-select" value={durationType} onChange={e =>
setDurationType(e.target.value)}>
      <option value="t">Ticks</option>
      <option value="s">Seconds</option>
      <option value="m">Minutes</option>
      <option value="h">Hours</option>
      <option value="d">Days</option>
    </select>
  </div>
</div>
<div className="payout-preview">
  <div>
    <div className="payout-label">Est. Payout</div>
    <div className="payout-val">${estimatedPayout}</div>
  </div>
  <div style={{ textAlign:"right" }}>
    <div className="payout-label">Stake</div>
    <div style={{ fontSize:14, fontWeight:700, color:"var(--text)"
}}>${parseFloat(stake||0).toFixed(2)}</div>
  </div>
</div>
<div className="contract-btns">
  <button className="contract-btn call" onClick={() => placeTrade("CALL")}>
    <span className="arrow">▲</span><span>RISE</span>
  </button>
  <button className="contract-btn put" onClick={() => placeTrade("PUT")}>
    <span className="arrow">▼</span><span>FALL</span>
  </button>
</div>
{!connected && <div style={{ fontSize:11, color:"var(--muted)",
textAlign:"center", lineHeight:1.6 }}>Enter your API token above to connect.</div>}
</div>
)}

{activeTab === "positions" && (
  <div className="open-contracts">

```

```

    {openContracts.length === 0 ? (
      <div className="empty">No open positions.<br />Place a trade to see it here.
    ) : openContracts.map(c => (
      <div className="contract-card" key={c.contract_id}>
        <div className="contract-card-header">
          <span className={`contract-tag ${c.contract_type === "CALL" ? "call" :
"put"}}`}>
            {c.contract_type === "CALL" ? "▲ RISE" : "▼ FALL"}
          </span>
          <span className={`contract-pnl ${((c.profit||0) >= 0 ? "pos" : "neg")}`} >
            {(c.profit||0) >= 0 ? "+" : ""}{(c.profit||0).toFixed(2)}
          </span>
        </div>
        <div className="contract-details">
          {[["Symbol",c.underlying],["Buy Price",`${c.buy_price?.toFixed(2)}`],
["Payout",`${c.payout?.toFixed(2)}`],["ID",`#${c.contract_id}`]].map(([k,v]) => (
            <div key={k}><div className="cd-item">{k}</div><div className="cd-val">
{v}</div></div>
          )))
        </div>
      </div>
    ))
  </div>
  {activeTab === "history" && (
    <div className="history-list">
      {history.length === 0 ? (
        <div className="empty">No trade history yet.<br />Connect your account to load
trades.</div>
      ) : history.map((t, i) => (
        <div className="history-item" key={i}>
          <div className="history-left">
            <div className="history-sym">{t.shortcode?.split("_")[0] || "-"}</div>
            <div className="history-meta">{t.transaction_time ? new
Date(t.transaction_time*1000).toLocaleString() : "-"}</div>
          </div>
          <div className={`history-profit ${((t.profit||0) >= 0 ? "pos" : "neg")}`} >
            {(t.profit||0) >= 0 ? "+" : ""}{(t.profit||0).toFixed(2)}
          </div>
        </div>
      )))
    </div>
  )}

  {activeTab === "account" && (
    <div className="account-panel">
      {!account ? (
        <div className="empty">Connect your API token<br />to view account details.

```

```

</div>
        ) : (
            <>
                <div className="account-card">
                    <div className="account-name">{account.fullname || account.loginid}</div>
                    <div className="account-balance">{account.currency}
{account.balance?.toFixed(2)}</div>
                    <div className="account-currency">{account.loginid} · {account.email}
</div>

                </div>
                <div className="account-grid">
                    {[["Login ID",account.loginid],["Currency",account.currency],
["Country",account.country?.toUpperCase()||"-"],["Scopes",account.scopes?.join(",
")||"-"]].map(([label,val]) => (
                        <div className="account-stat" key={label}>
                            <div className="account-stat-label">{label}</div>
                            <div className="account-stat-val" style={{ fontSize: label==="Scopes"?
10:14 }}>{val}</div>
                        </div>
                    )]}
                </div>
            </>
        )}
    </div>
    )}
</div>
    </div>
    {toast && <div className={`toast ${toast.type === "error" ? "error" : ""}`}>{toast.msg}
</div>}
</>
    );
}

```